Practical 11 --- A Robot That Checks Your Wor

Continuous integration with GitHub Actions, rehearsed on your own machine

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L20-L21
Course Outcome	CO4
Lab duration	120 minutes
Memory needed	about 350 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Explain what continuous integration is and which problem it solves.
2. Write a GitHub Actions workflow that installs, tests and checks a project.
3. Run the pipeline's own steps on your machine before pushing anything.
4. Read a failed pipeline and say exactly which step broke and why.

What you need to know first

Picture the Friday-evening deploy. Someone pulls the latest code, runs the tests on their laptop, green, and ships. On Monday it is broken, because their laptop had a library the server does not because two of the tests were never actually run.

Continuous integration, usually written **CI**, removes the human from that sentence. Every time pushes code, a clean machine somewhere fetches it, installs everything from scratch, runs every and reports pass or fail. No one has to remember. No one can skip it.

The vocabulary is small.

A **workflow** is the whole automated procedure, written as a YAML file that lives in your repository `.github/workflows/`.

A **trigger** says when to run it --- typically on every push, and on every pull request.

A **job** runs on a fresh machine. A workflow can have several, and they can run at the same time.

A **step** is one command inside a job. Steps run in order, and the first one to fail stops the job.

That last rule is the whole mechanism. Remember from P04 and P05 that a program returns an **code**: 0 for success, anything else for failure. `pytest` returns 1 if any test failed. The pipeline

looks at that number. Your tests from P05 are what makes CI worth having --- without them the r
nothing to check.

Today you will write a real workflow file, and then run its steps on your own machine with a small
we build together. Rehearsing locally is a genuinely professional habit: pushing a broken workflo
waiting three minutes to find out is a slow way to work.

Before you start

- Practical P05 is finished. You can write tests and run pytest.
- You have seen YAML twice now. Spaces only.
- A GitHub account is useful for the optional last step, but nothing today requires one.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the ne
number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell c
you jump ahead you will see a `NameError`, which just means "you have not made that thing y

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then
again from the first cell. Nothing is damaged by doing this.

In [ ]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['yaml', 'pytest']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['yaml', 'pytest']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the fil
there.

In [ ]:
```python
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Build a small project worth checking

CI needs something to test. We reuse the shape from P05: a module, some tests, and a requirer file.

In [ ]:
```
from pathlib import Path

WORK = Path("work")
WORK.mkdir(exist_ok=True)

(WORK / "orders.py").write_text('''"Helpers for delivery orders."

TRAFFIC_LEVELS = (1, 2, 3)


def minutes_per_km(delivery_min, distance_km):
    "How many minutes each kilometre took."
    if distance_km <= 0:
        raise ValueError("distance_km must be positive")
    return delivery_min / distance_km


def is_valid_order(order):
    "True if this order is usable for training."
    return (order.get("distance_km", 0) > 0
            and order.get("prep_time_min", 0) >= 0
            and order.get("traffic_level") in TRAFFIC_LEVELS
            and order.get("rain") in (0, 1))
''', encoding="utf-8")

(WORK / "test_orders.py").write_text('''"Tests for orders.py."

import pytest

from orders import is_valid_order, minutes_per_km


def test_minutes_per_km():
    assert minutes_per_km(30, 10) == 3.0


def test_zero_distance_raises():
    with pytest.raises(ValueError):
        minutes_per_km(30, 0)


def test_good_order():
    assert is_valid_order({"distance_km": 5.0, "prep_time_min": 20,
                           "traffic_level": 2, "rain": 0})
```

```
def test_bad_traffic():
    assert not is_valid_order({"distance_km": 5.0,
                               "prep_time_min": 20,
                               "traffic_level": 9, "rain": 0})
''', encoding="utf-8")

(WORK / "requirements.txt").write_text(
    "pytest==9.1.1\nscikit-learn==1.9.0\npandas==2.3.3\n",
    encoding="utf-8")

for f in sorted(WORK.glob("*")):
    if f.is_file():
        print("  ", f.name)
```

Step 2 --- Write the workflow

This is a real GitHub Actions file. Read it as four questions: **when** does it run, **where** does it run, does it do, and **in what order**.

`uses:` pulls in a ready-made action somebody else wrote --- `checkout` fetches your code, `python` installs Python. `run:` is your own shell command.

The `name:` on each step is what you will see in the web interface, so make them say somethin

In [ ]:
```
workflow = """name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Get the code
        uses: actions/checkout@v4

      - name: Install Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install the libraries
        run: pip install -r requirements.txt

      - name: Run the tests
        run: pytest -q

      - name: Check the config file is valid
        run: python -c "import yaml, pathlib; yaml.safe_load(pathlib.Path
"""

flow_dir = WORK / ".github" / "workflows"
flow_dir.mkdir(parents=True, exist_ok=True)
(flow_dir / "ci.yml").write_text(workflow, encoding="utf-8")
```

```
print(workflow)
```

`runs-on: ubuntu-latest` is worth noticing. The pipeline runs on Linux, on a machine crea seconds ago with nothing installed. That is the point: if it works there, it does not depend on anyt peculiar about your laptop.

Step 3 --- Check the workflow is valid YAML before pushing it

A workflow with a YAML error does not fail loudly --- GitHub often just ignores it, and you wonder nothing ran. Check it yourself.

One oddity to know about: in YAML, a bare `on` is read as the boolean `True`. So after parsing, trigger section appears under the key `True`, not `"on"`. That surprises everybody once.

In [ ]:
```
import yaml

with open(flow_dir / "ci.yml", encoding="utf-8") as fh:
    flow = yaml.safe_load(fh)

print("top-level keys:", list(flow))

triggers = flow.get("on", flow.get(True))
print("triggers      :", list(triggers))
print("jobs          :", list(flow["jobs"]))

steps = flow["jobs"]["test"]["steps"]
print()
print(f"{len(steps)} steps:")
for i, step in enumerate(steps, 1):
    kind = "uses" if "uses" in step else "run"
    print(f"  {i}. [{kind:4}] {step.get('name', '(unnamed)')}")
```

Step 4 --- Build a runner for the steps you can rehearse

`uses:` steps need GitHub's machinery, so we skip those. But every `run:` step is just a shell command --- and those you can rehearse here, in order, stopping at the first failure, exactly as th pipeline would.

One adjustment. On the pipeline's fresh Linux machine, `python` and `pip` mean the right thin your laptop `python` may be a different installation altogether --- on Windows it is often a stub f Microsoft Store. So the runner rewrites those three words to point at the exact interpreter runnin notebook.

That difference **is** the lesson: your machine is not the pipeline's machine, which is why the pipeli exists.

In [ ]:
```
import subprocess
import sys

HERE_PY = f'"{sys.executable}"'
TRANSLATIONS = [
    ("pip", f"{HERE_PY} -m pip"),
```

```
    ("pytest", f"{HERE_PY} -m pytest"),
    ("python", HERE_PY),
]


def for_this_machine(command):
    """Rewrite a CI command so it uses this notebook's Python."""
    for word, replacement in TRANSLATIONS:
        if command == word or command.startswith(word + " "):
            return replacement + command[len(word):]
    return command


def run_pipeline(steps, cwd=WORK):
    """Run the `run:` steps in order and stop at the first failure."""
    results = []
    for step in steps:
        name = step.get("name", "(unnamed)")
        if "run" not in step:
            print(f"[skip] {name}  (needs GitHub: {step['uses']})")
            results.append({"name": name, "status": "skipped"})
            continue

        command = step["run"].strip()
        print(f"[run ] {name}")
        print(f"       $ {command}")
        done = subprocess.run(for_this_machine(command), cwd=cwd,
                              shell=True, capture_output=True,
                              text=True)
        ok = done.returncode == 0
        tail = (done.stdout + done.stderr).strip().splitlines()[-4:]
        for line in tail:
            print("       |", line)
        print(f"       -> exit {done.returncode} "
              f"{'PASS' if ok else 'FAIL'}")
        results.append({"name": name,
                        "status": "pass" if ok else "fail",
                        "exit_code": done.returncode})
        if not ok:
            print()
            print("PIPELINE FAILED. Later steps do not run.")
            break
        print()
    return results


print("runner ready")
```

Step 5 --- Rehearse the pipeline

Run it. The install step uses the libraries you already have, so it is quick; the test step is the one matters.

The last step will fail --- there is no `config.yaml` in this folder. That is deliberate. A pipeline th never failed is a pipeline you have not tested.

In [ ]:
```
results = run_pipeline(steps)
```

```
print("=" * 55)
for r in results:
    print(f"  {r['status']:<8} {r['name']}")
```

Read the failure. It names the step, shows the command, and shows the error. That is exactly wh red cross in GitHub's web interface will show you --- so you can debug it here, in seconds, instea pushing and waiting.

Step 6 --- Fix the pipeline

The pipeline asked for a `config.yaml` that does not exist. Two honest fixes: create the file, o asking for it. We create it, because a real project would have one.

In [ ]:
```
(WORK / "config.yaml").write_text(
    "split:\n  test_size: 0.2\n  seed: 42\n", encoding="utf-8"
)

results = run_pipeline(steps)

print("=" * 55)
passed = sum(1 for r in results if r["status"] == "pass")
failed = sum(1 for r in results if r["status"] == "fail")
print(f"  {passed} passed, {failed} failed")
print("  PIPELINE GREEN" if failed == 0 else "  PIPELINE RED")
```

Step 7 --- Now break a test and watch the robot catch it

This is the whole value of CI in one cell. Introduce a bug the way a tired person would --- a wron operator --- and let the pipeline find it.

In [ ]:
```
original = (WORK / "orders.py").read_text(encoding="utf-8")
(WORK / "orders.py").write_text(
    original.replace("return delivery_min / distance_km",
                     "return delivery_min * distance_km"),
    encoding="utf-8",
)

print("A bug has been introduced. Running the pipeline...")
print()
broken_results = run_pipeline(steps)

(WORK / "orders.py").write_text(original, encoding="utf-8")
print()
print("bug removed, orders.py restored")
```

The pipeline went red at the test step, and the steps after it never ran. On GitHub this is the poin the pull request is blocked and the author gets an email. Nobody had to notice.

Step 8 --- Make the pipeline faster with a cache

Installing libraries from scratch on every push is slow. GitHub can keep pip's download cache be runs; `setup-python` does it with one extra line.

Speed is not vanity here. A pipeline that takes fifteen minutes is a pipeline people learn to ignore

In [ ]:
```
cached = yaml.safe_load(
    (flow_dir / "ci.yml").read_text(encoding="utf-8")
)
for step in cached["jobs"]["test"]["steps"]:
    if step.get("uses", "").startswith("actions/setup-python"):
        step["with"]["cache"] = "pip"

(flow_dir / "ci-cached.yml").write_text(
    yaml.safe_dump(cached, sort_keys=False), encoding="utf-8"
)

print(yaml.safe_dump(cached["jobs"]["test"]["steps"][1],
                     sort_keys=False))
```

Step 9 --- What happens on GitHub

You do not have to do this today, but you should know the shape of it.

In [ ]:
```
print("To make this real:")
print()
print("  1. Put your project in a GitHub repository.")
print("  2. Commit the file at .github/workflows/ci.yml")
print("     -- the path must be exactly that.")
print("  3. git push")
print("  4. Open the repository in a browser and click the")
print("     'Actions' tab at the top.")
print()
print("What you will see first: a list of workflow runs, newest")
print("at the top, each with a yellow dot (running), a green tick")
print("(passed) or a red cross (failed). Click a run, then a job,")
print("to expand each step and read its output.")
print()
print("In Settings -> Branches you can then add a rule that")
print("REQUIRES this check to pass before anyone may merge into")
print("main. That is the moment CI stops being advice and starts")
print("being a rule.")
```

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you whic are correct. You can run it as many times as you like.

Task T1 --- Trigger it on the right events

Build a dictionary `T1_triggers` describing a workflow that runs:

- on every **push** to the branches `main` **and** `develop`
- on every **pull request** targeting `main`

The shape must match what GitHub expects, that is: `{"push": {"branches": [...]}, "pull_request": {"branches": [...]}}`

> **Hint.** This is a plain Python dictionary; no YAML needed. Look at the `on:` block printed Step 2 and write the same structure with one extra branch under `push`.

In [ ]:
```
# TODO: push on main and develop, pull_request on main
T1_triggers = None

print(T1_triggers)
```

Task T2 --- Add a step that would have caught a real bug

Add one more step to the workflow, named exactly `Check the model can be imported`, runs the command:

```
python -c "import orders; print(orders.minutes_per_km(30, 10))"
```

Append it to the `steps` list, write the updated workflow back to `work/.github/workflows/ci.yml`, and store the new steps list in `T2_steps`.

> **Hint.** A step is a dictionary with `name` and `run` keys. Use `steps.append({...})`, then rebuild the file with `yaml.safe_dump(flow, sort_keys=False)`. Set `T2_steps = steps`.

In [ ]:
```
new_step = {
    "name": "Check the model can be imported",
    # TODO: the command to run
    "run": None,
}

# TODO: append it to steps, save the workflow file, and set T2_steps
T2_steps = None

print(T2_steps)
```

Task T3 --- Rehearse it, and get it green

Run your six-step pipeline with `run_pipeline` and store the result list in `T3_results`.

Every step that actually runs must pass. If something fails, read the output, fix it, and run again - as you would before pushing.

> **Hint.** `T3_results = run_pipeline(T2_steps)`. If the import check fails with `ModuleNotFoundError`, check that `orders.py` is still intact -- Step 7 modified it a put it back.

In [ ]:
```
# TODO: run the pipeline and keep the results
T3_results = None

print(T3_results)
```

Self-check

Run the cell below to mark your work.

In [ ]:
```
# ---------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ---------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | both trigger kinds are present', lambda: set(T1_triggers) ==
_check('T1 | push covers main and develop', lambda: sorted(T1_triggers['p
_check('T1 | pull_request targets main only', lambda: T1_triggers['pull_r
_check('T2 | the workflow now has six steps', lambda: len(T2_steps) == 6)
_check('T2 | the last one is the import check, with a run command', lambd
_check('T2 | its command really imports orders and calls the function', l
_check('T2 | the change was saved to the workflow file', lambda: 'Check t
_check('T3 | the pipeline ran all six steps', lambda: len(T3_results) ==
_check('T3 | nothing failed', lambda: not any(r['status'] == 'fail' for r
_check('T3 | the tests and the import check both actually ran', lambda: s

print("=====================================================================
print("SELF-CHECK   Practical 11 --- A Robot That Checks Your Work")
print("=====================================================================
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("---------------------------------------------------------------------
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("=====================================================================
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
```

```
else:
    print("Read the FAIL lines above, fix those tasks, run this cell agai
```

What to submit

1. This notebook, with every cell run and its output visible.
2. Your `work/.github/workflows/ci.yml`.
3. In a markdown cell, two sentences: what the pipeline did when you broke the test, and why after it did not run.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, pipeline goes green then red	3
Task T1 --- correct triggers on the workflow	2
Task T2 --- a useful step added to the pipeline	3
Task T3 --- the pipeline rehearsed and green	2
Total	**10**

Read more

- GitHub Docs --- Understanding GitHub Actions --- https://docs.github.com/en/actions/learn-g actions/understanding-github-actions
- GitHub Docs --- Workflow syntax --- https://docs.github.com/en/actions/reference/workflow-s for-github-actions
- GitHub Docs --- Building and testing Python --- https://docs.github.com/en/actions/automatir and-tests/building-and-testing-python
- GitHub Docs --- About protected branches --- https://docs.github.com/en/repositories/configu branches-and-merges-in-your-repository/managing-protected-branches/about-protected-bra